



Why Python? A Comprehensive Guide

Overview:

Your journey to mastering Python programming starts here

Topics:

Syntax, Data Types, Functions, OOP & More



Ambreen Abdul Raheem
Data Analyst & Web Developer

Python is a Programming and Scripting Language



- Python supports both programming (building applications) and scripting (automating tasks).
- Used for web development, data science, automation, AI, and more.
- Enables quick prototyping and large-scale system development.
- “Python is a high-level, interpreted programming language used for both general-purpose programming and scripting.”

Agenda



- Python is a Programming and Scripting Language
- Syntax, Case Sensitivity, and Comments in Python
- Operators and Punctuators
- Python is a Typed Language
- Expression Execution
- Input in Python
- Data Types in Python
- Conditional Statements and Indentation
- Indexing and Slicing
- Loops in Python in Detail
- Functions in Python in Detail
- Object-Oriented Programming (OOPs) in Python in Detail

Operators and Punctuators In Python



- Arithmetic: +, -, *, /, %, ** (exponent), // (floor division)
- Assignment: =, +=, -=, *=, /=
- Comparison: ==, !=, >, <, >=, <=
- Logical: and, or, not
- Punctuators: :, ., ,, (,), [], {}
- Operators build expressions
- Punctuators structure code blocks
- Precedence determines evaluation order (PEMDAS)

Python is a Typed Language



- Python variables have types but are dynamically assigned at runtime, offering flexibility without explicit declarations.
- Strong typing means types are enforced—no implicit mixing of incompatible types without conversion.
- Type conversion options: implicit (automatic, limited) vs explicit using `int()`, `float()`, `str()`, `bool()` functions.

Expression Execution



- Python uses PEMDAS order: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction for evaluating expressions.
- Assignment operations are evaluated right-to-left, allowing chained assignments like `a = b = c = 5`.
- Expressions are evaluated stepwise from highest to lowest precedence, ensuring predictable and consistent results.

Input in Python



- The `input()` function reads user input as a string by default.
- Convert input to required types using casting functions: `int()`, `float()`, or `bool()`.
- Example: `age = int(input("Enter your age: "))`
prompts the user and converts the response to an integer.

Data Types in Python



- str: text strings (e.g., "Hello")
- int: whole numbers (e.g., 42, -7)
- float: decimal numbers (e.g., 3.14, -0.5)
- bool: True or False values
- None: represents null or no value
- list: ordered, mutable collection [1, 2, 3]
- tuple: ordered, immutable collection (1, 2, 3)
- dict: key-value pairs {"name": "Python"}
- set: an unordered collection of unique elements {1,2,3}

Conditional Statements and Indentation



- Python uses `if`, `elif`, and `else` keywords to control program flow based on conditions, allowing code to make decisions and execute different blocks.
- Indentation (typically 4 spaces) defines code blocks instead of braces—Python enforces this structure, making proper indentation essential for code correctness.
- Nested conditions and multiple `elif` branches enable complex decision trees while maintaining clean, readable code structure.

Indexing and Slicing



- Indexing accesses elements by position using zero-based numbering; negative indices count from the end (e.g., -1 is the last element).
- Slicing syntax `[start:stop:step]` extracts subsequences. Start is inclusive; stop is exclusive; step controls direction and interval.
- Works seamlessly on strings, lists, and tuples, enabling powerful data extraction and manipulation.

Loops in Python



- for loops iterate over sequences or range()
- while loops run while condition is True
- break exits the loop early
- continue skips current iteration
- else clause runs if loop completes normally
- range() generates number sequences
- Nested loops for multi-dimensional iteration
- Loop optimization for better performance

Functions in Python



- `def` keyword to define functions
- Positional, keyword, and default parameters
- `*args` and `**kwargs` for flexible arguments
- `return` statement outputs values
- Local vs global variable scope
- Lambda functions (anonymous)
- Recursion basics
- Common built-in functions

Object-Oriented Programming



- Classes and Objects
- Attributes and Methods
- **init** Constructor
- Encapsulation (`_` prefix)
- Inheritance Types
- Polymorphism & Overriding
- Abstraction
- Dunder Methods

Thank You!
Start your Python
journey today

